

DSA GRAPH

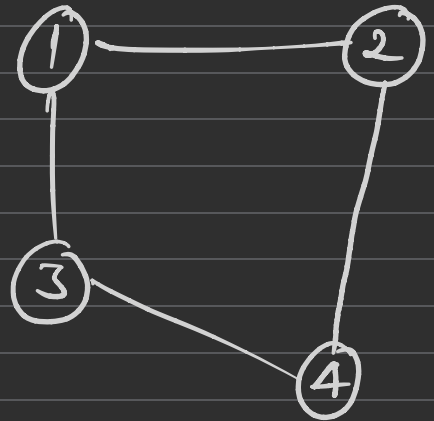
A person wearing a pink beanie and a blue shirt is pointing their right index finger at a whiteboard. The whiteboard is filled with various diagrams, including a tree structure, a graph, and some code snippets. The background is dark and out of focus.

What is Graph

- Non-linear Data structure
- Nodes and Edges { vertices and edges }

collection of set of vertices (nodes)
and edges

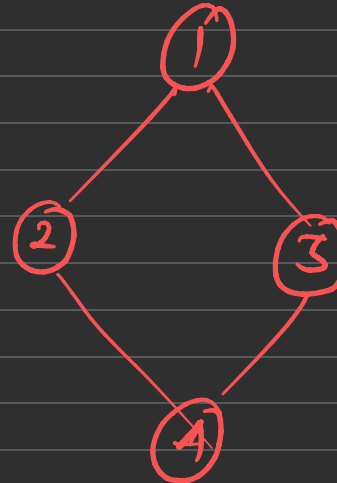
$$G \in (V, E)$$



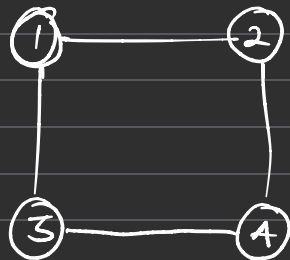
Tree (Never contain
cycle)



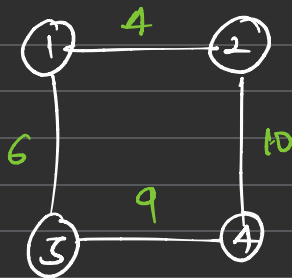
graph (can contain
cycle)



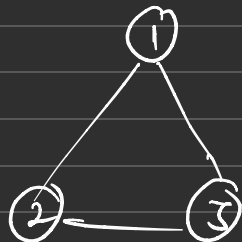
Types of Graphs



Simple graph

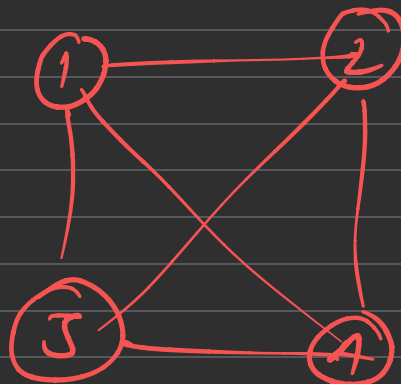


Weighted graph



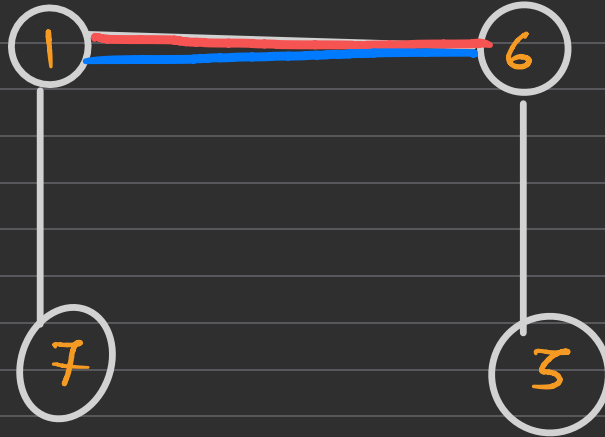
Cycle graph

Complete graph



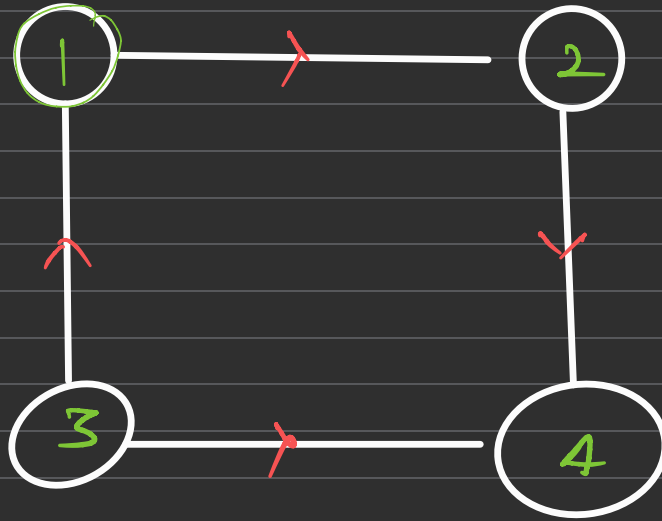
Directed Undirected Graph

* Undirected graph
graph whose edge don't have
particular direction



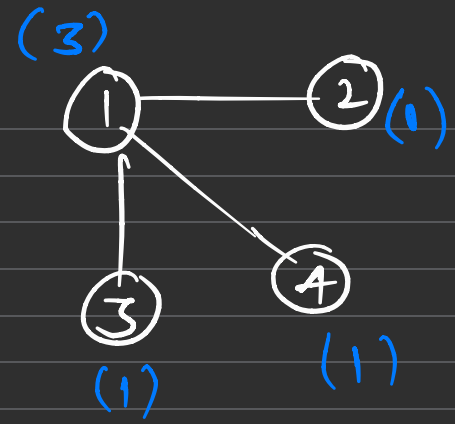
Directed Graph

graph whose edge have particular direction



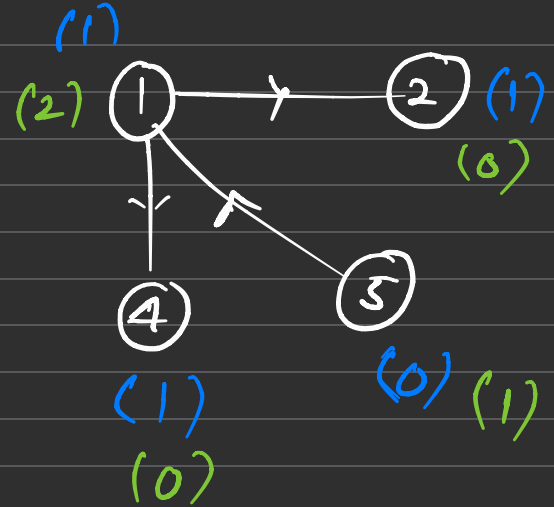
* Degree (Undirected)

No. of edge connected to a Node



* Indegree (Directed)

No. of Edges coming inside a Node



* Outdegree (Directed)

No. of Edges going outside a Node

How to represent the Graph

✓ Adj. Matrix

✓ Adj. List

Adj. Matrix

$$V = 4$$

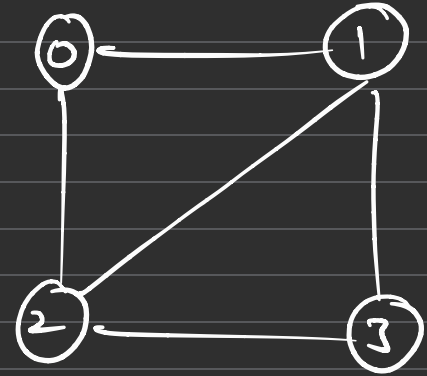
Build the matrix of $V \times V$

col

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

Row

mat

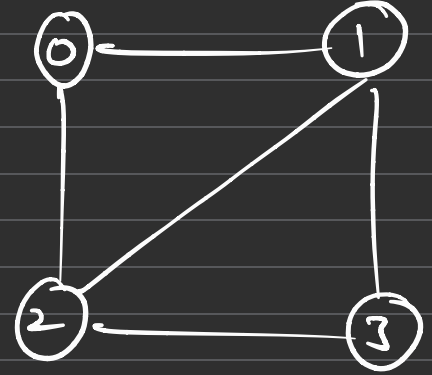


$mat[i][j] = 1$
iff there is edge
from i to j
else
 $mat[i][j] = 0$

Adj. List

$$V = 4$$

0	1	2
1	(0 2 3)	
2	(0, 1, 3)	
3	(1, 2)	



$$(0) \rightarrow (1, 2)$$

$$(1) \rightarrow (0, 3, 2)$$

$$(2) \rightarrow (0, 1, 3)$$

$$(3) \rightarrow (1, 2)$$

Graph Traversal

✓ DFS

BFS

DFS { Pre
Post
In

Not standard
for tree

Level } BFS

0	1	2	3
1	4	5	6
2	7	8	9

```
for (auto i : mat[index]) (cpp)
{
    print(i)
}
```

index = 1

```
for (Integer i : mat.get(index)) (java)
{
    print(i)
}
```

arr

6	7	2	1
0	1	2	3

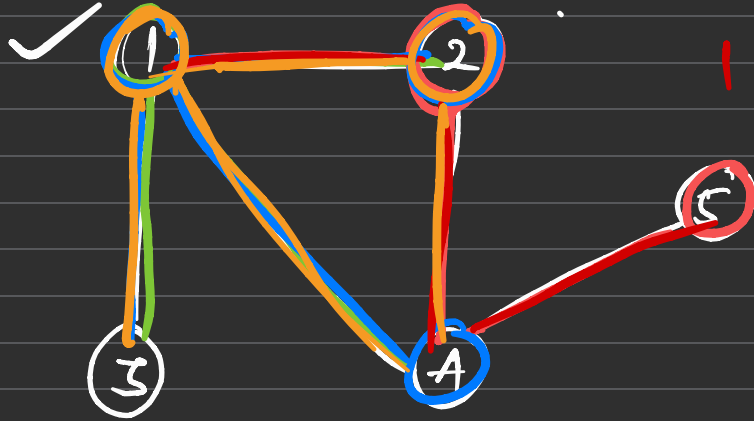
```
for (int i : arr)
    println(i)
```

```
for (auto it : arr)
    print(it)
```

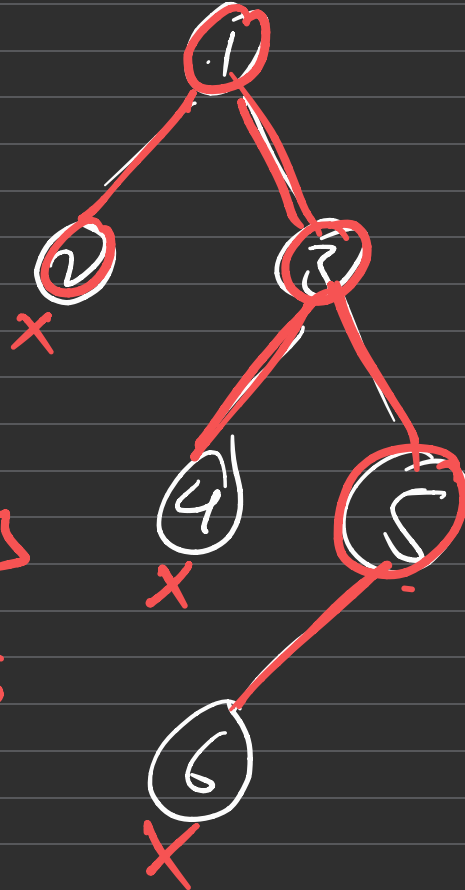
print(arr[2]) = 2

1 2 3 4 1 4 5 1 2 3 4

Re



1 2 4



1

1 2 3

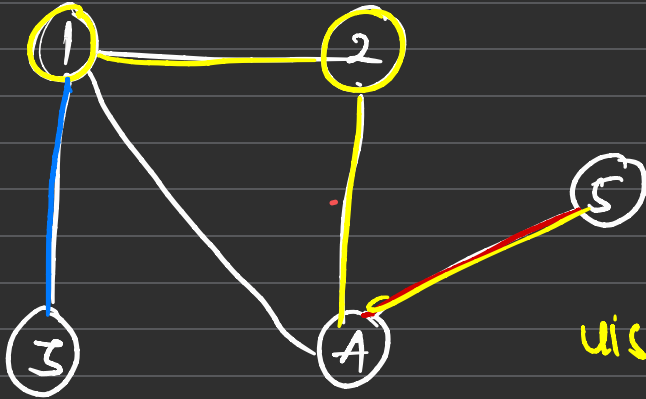
4 5 6

Stop



stop point?

Node values = Index



we need to maintain array, (visited array)

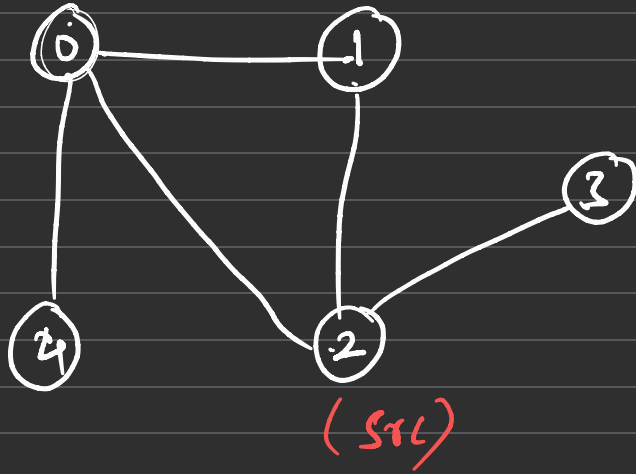
visited

X	1	1	0 1	0 1	1
0	1	2	3	4	5

1 2 4 5 3

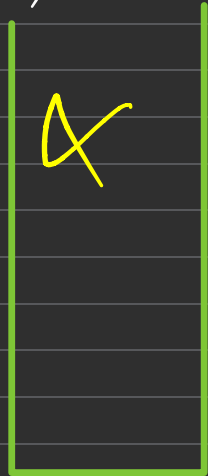
0 = Not Visited
1 = visited

BFS (Breadth First Search → level order)



0	*	2	4
1	0	2	
2	0	1	3
3	2		
4	0		

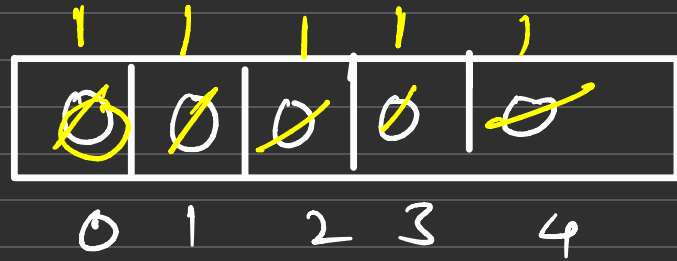
Adj list



Queue
<int>

2 0 1 3 4

visit



```
void BFS ( vector<int> adj[], v )  
{
```

```
    queue<int> q
```

```
    int visit [v] = { 0 }
```

```
    q.push ( 0 )
```

```
    visit[0] = 1
```

```
while( !q.empty() )  
{  
    int node = q.front()  
    q.pop()
```

```
    print ( node)
```

[1|2|4]

```
    for ( auto it : adj[ node] )
```

```
    {  
        if ( visit(it) == 0 )
```

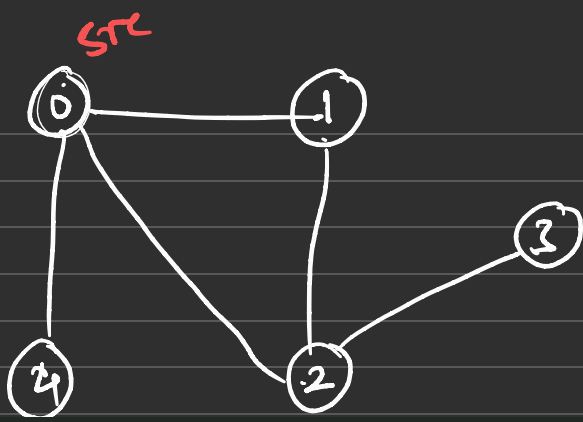
```
        {  
            q.push (it)
```

```
            visit(it) = 1
```

}

}

}



0	1	2	4
1	0	2	
2	0	1	3
3	2		
4	0		

Adj list



```

while(!q.empty()){
    int node = q.front();
    q.pop();

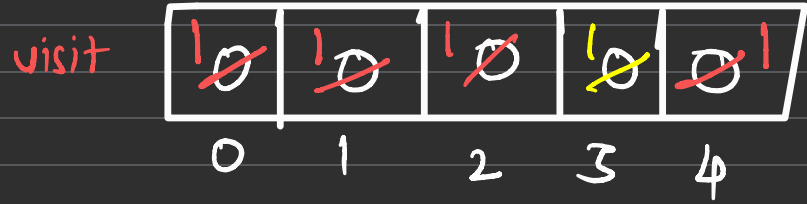
    ans.push_back(node);

    for (auto adjNode: adj[node]){
        // not visited
        if (visit[adjNode] == 0){
            q.push(adjNode);
            visit[adjNode] = 1;
        }
    }
}

return ans;

```

0 1 2 4 3



node = 3

adjNode = 2

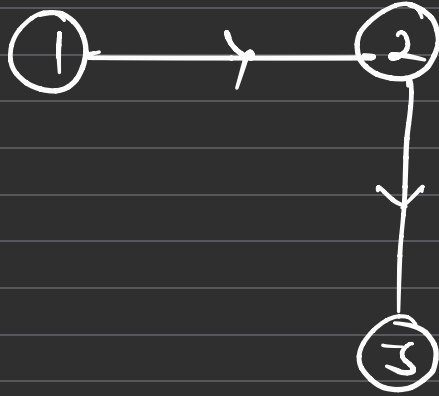
- Topological sort

- linear ordering sorting technique which mean, if there is edge from $u \rightarrow v$, then in your sorting u should come before v

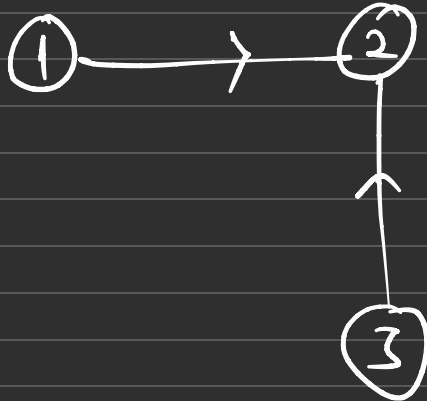
- Only applied on

1. Directed graph

2. Acyclic graph



1 2 3



1 3 2

Why not on Undirected graph

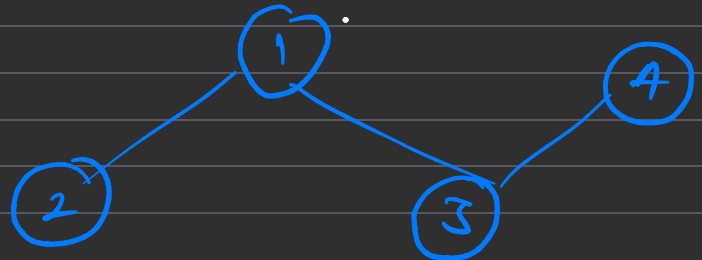


— Indegree (Directed)

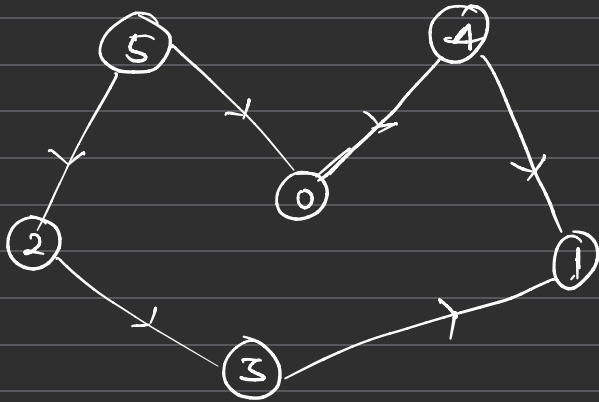
No. of edges coming inside

— Outdegree (Directed)

— degree (Undirected)



- find topo. sort



- find indegree

- Take the Node with mini. indegree
& print it.

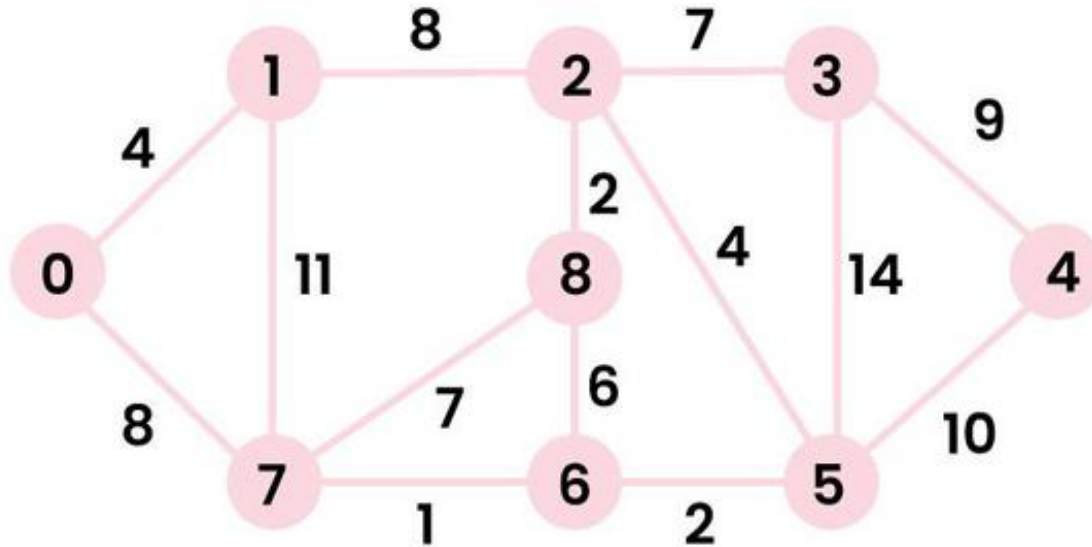
And Remove it, Out going edges

— Dijkstra's Algorithm:

— Single source shortest path

— Negative edge weight $\rightarrow$ does not work

_ Dijkstra Algorithm



Working of Dijkstra's Algorithm



